

“Kernel Monitor”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**Chaitanya
Bhardwaj
Salman**

**2520090094
2520090182
2520089070**

Under the Esteemed Faculty Guidance

Mrs. Alekhya Dasari
Operating Systems and Systems Programming (25CS2104E)



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Kernel Monitor” is a record of Bonafide work of Chaitanya - 2520090094, Bhardwaj - 2520090182, Salman - 2520089070 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

The project is developed as part of the Operating Systems and Systems Programming (25CS2104E) course during the academic year 2026–27, Term-I.

**Chaitanya
Bhardwaj
Salman**

**2520090094
2520090182
2520089070**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled “Kernel Monitor” is a bonafide work done and submitted by Chaitanya – 2520090094, Bhardwaj – 2520090182, and Salman – 2520089070, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year 2026–27, Term-I.

The project is completed under the Operating Systems and Systems Programming (25CS2104E) course.

Faculty Signature

Course Coordinator Signature

HOD Signature

ACKNOWLEDGEMENT

The successful completion of this project was made possible through the guidance, support, and cooperation received during the development of the Kernel Monitor. We sincerely thank everyone who contributed directly or indirectly to the completion of this project.

We express our sincere gratitude to Mrs. Alekhya Dasari for her faculty guidance and support throughout the project. Her guidance helped us understand and apply Operating Systems and Systems Programming concepts in the development of the Kernel Monitor.

We also thank the teaching and non-teaching staff for providing the necessary academic support and facilities. Finally, we sincerely thank our parents, friends, and classmates for their encouragement and cooperation during the project.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Chaitanya 2520090094

Bhardwaj 2520090182

Salman 2520089070

Abstract

The Kernel Monitor is a Linux-based system programming project designed to monitor and display important information about the operating system and its running processes. The project addresses the need for a simple monitoring tool that helps users understand system resource utilization, process states, CPU usage, memory usage, and other kernel-level information. The project is closely related to Operating Systems concepts such as process management, CPU scheduling, memory management, system calls, process states, and resource monitoring.

The proposed solution will be implemented in C/C++ on Linux using POSIX APIs and Linux interfaces such as the /proc filesystem and system calls. The monitor will periodically collect system information and present it in a clear and readable format. It can display details such as process ID (PID), process state, CPU utilization, memory consumption, uptime, and overall system statistics.

The project provides practical experience with Linux system programming and demonstrates how user-level programs can interact with operating-system kernel information. The expected outcome is a lightweight and user-friendly Kernel Monitor capable of providing real-time or periodic information about system and process activities.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		—
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		—
		2.1 Process Management	
		2.2 CPU Scheduling and Process States	
		2.3 Memory Management and Resource Monitoring	
		2.4 System Calls and /proc Filesystem	
3	System Requirements		—
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Linux / Ubuntu Environment	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		—
		4.1 C / C++ Programming	
		4.2 Linux / Ubuntu	
		4.3 /proc Filesystem	
		4.4 POSIX APIs and System Calls	
		4.5 VS Code / Vim / Nano	
		4.6 Git / GitHub	
5	System Architecture and Design		—
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Data Collection and Processing	

		5.4 Process Monitoring Module	
		5.5 Input and Output Design	
6	Implementation		—
		6.1 System Information Collection	
		6.2 Process Information Collection	
		6.3 CPU and Memory Monitoring	
		6.4 /proc Filesystem Handling	
		6.5 Periodic Refresh Mechanism	
		6.6 Error Handling	
7	Execution and Demonstration		—
		7.1 Compilation in Linux / Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		—
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Expected Results	
		8.4 Skills and Learning Outcomes	
9	References		—
		9.1 Linux / GNU Documentation	
		9.2 POSIX / Linux API Documentation	
		9.3 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Modern operating systems manage a large number of processes and system resources simultaneously. Monitoring CPU utilization, memory consumption, process states, system uptime, and other system activities is important for understanding how the operating system is functioning. The Kernel Monitor project is designed as a Linux-based system programming application that provides users with a simple way to observe important operating-system and process information.

1.2 Problem Statement

Users often need to understand which processes are running and how system resources are being utilized. The Kernel Monitor addresses this requirement by collecting system and process information and presenting it in a clear and readable format. The project demonstrates how a user-space application can obtain information maintained by the Linux kernel through the /proc virtual filesystem and Linux/POSIX APIs.

1.3 Scope of the Project

The scope of the project is to develop a lightweight Linux-based monitoring application using C/C++. The monitor focuses on system and process information such as CPU utilization, memory usage, system uptime, process IDs, process states, process memory consumption, and CPU-related process information. The project also includes periodic refreshing so that changes in system activity can be observed.

1.4 Objectives of the Project

The main objectives of the Kernel Monitor are to develop a Linux-based monitoring application using C/C++, monitor important system resources such as CPU usage, memory usage and system uptime, retrieve information about running processes, and understand Linux system calls, POSIX APIs and the /proc filesystem used for accessing operating-system information.

1.5 Importance of the Application

The Kernel Monitor provides practical experience with Linux system programming and Operating Systems concepts. It demonstrates the relationship between user-space applications and information provided by the Linux kernel. The project is useful for understanding process management, resource monitoring, system calls, Linux APIs, file handling, process identification and kernel-provided information.

1.6 Target Users / Audience

The project is intended for students and learners studying Operating Systems and Systems Programming, as well as users who need a simple Linux-based view of system and process activity. It can be used as a practical demonstration of how operating-system information can be collected and displayed by a user-space program.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project is designed to run on a standard computer capable of running a Linux environment. A basic processor, sufficient RAM for normal Linux operation, and adequate storage for the source code and development tools are required. The project does not require specialized hardware.

2.2 Software Requirements

The proposed Kernel Monitor uses Linux/Ubuntu as the operating-system and execution environment. C/C++ is used for implementation. The Linux /proc filesystem provides kernel and process information. Development may be performed using VS Code, Vim or Nano, while Git/GitHub can be used for source-code management and collaboration.

2.3 Operating System Environment – Ubuntu / Linux

The application is intended to execute in a Linux environment such as Ubuntu. Linux provides the /proc virtual filesystem and system interfaces required to obtain information about processes and system resources. The environment also provides the compiler, system utilities and POSIX/Linux APIs needed for development and execution.

2.4 Development Tools and Compilers

The Kernel Monitor can be developed using C/C++ and compiled using a suitable Linux compiler such as GCC. Source-code editing can be performed using VS Code, Vim or Nano. Git and GitHub can be used to maintain the project source code and support collaboration.

2.5 Linux /proc Filesystem Requirements

The /proc filesystem is a central interface used by the project to obtain kernel-provided information. The monitor can scan process directories and read files such as /proc/[PID]/stat and /proc/[PID]/status to obtain process state, CPU-related statistics and memory information.

2.6 Functional Requirements

The system should collect general system information, identify running processes, obtain process-related details, calculate or present resource-usage information, display the collected results in a structured format, and support periodic refreshing of the displayed information. The application should also handle cases where a process terminates while its information is being read.

2.7 Non-Functional Requirements

The application should be lightweight, readable and suitable for execution in a Linux environment. The displayed information should be organized clearly so that users can understand system and process activity. The monitor should provide timely updates through its periodic refresh mechanism and should handle unavailable process information without terminating unexpectedly.